Web Application Requirements Specification and Analysis App

Sprint Report

Author:

Grigorios – Rafail Vasileiou

University of Ioannina

2026

VERSIONS HISTORY

Date	Version	Description
20/03/2026	1	Project setup and creation of base Spring Boot structure.
20/03/2026	2	Implementation of User entity and related functionality.
26/03/2026	3	Implementation of Project entity and its basic relationships and frontend design.
02/04/2026	4	Development of UseCase entity, service, Controller and frontend design.
14/04/2026	5	Implementation of CRCCard entity, Service and Controller and frontend design.
20/04/2026	6	Implementation of UML diagram generator services (PlantUML Use Case & CRC diagrams).
29/04/2026	7	Unit testing of service layer using JUnit and Mockito.
09/05/2026	8	Final version: bug fixes, full integration and completion of documentation and UML diagrams.

1 Introduction

1.1 Purpose

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies the this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project. At the last page, we highlight what needs to be modified in order for the project to run properly in every machine.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Apostolos Zarras
Scrum Master	Grigoris – Rafail Vasileiou
Development Team	Grigoris – Rafail Vasileiou

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	**20/03/2026**	**24/03/2026**	**1**	**US1, US2, US3**
2	**26/03/2026**	**31/03/2026**	**1**	**US4, US5, US6**
3	**02/04/2026**	**11/04/2026**	**2**	**US7, US8, US9, US10**
4	**14/04/2026**	**18/04/2026**	**1**	**US11, US12, US13, US14**
5	**20/04/2026**	**06/05/2026**	**2**	**US15, US16**

3 Use Cases

3.1 <Use Case 1>

Use case ID	CreateAccount
Actors	Developer
Pre conditions	A user has access to the application.
Main flow of events	1. The use case begins when the developer chooses to create an account or to log in so that he can securely access his workspace. 2. If the user chooses to create an account("register" option) 2.1. The user sets his username, email, password. 2.2. The system verifies his credentials and goes to log in page. 3. The user logs in to his account. 4. Use case ends when user logs out.
Post conditions	User has logged into his account successfully.

3.2 <Use Case 2>

Use case ID	ManageProject
Actors	Developer
Pre conditions	The developer is logged into his account.
Main flow of events	1. The use case begins when the developer chooses to view his projects with the "My Projects" button. 2. If the developer clicks the "Create" button, the system creates a new project.

	3. If there are existing projects: 3.1. The developer can choose to view a project, where he can organize the use cases and CRC cards. 3.2. The developer can choose the “Delete” option and the system deletes the corresponding project.
Post conditions	-

3.3 <Use Case 3>

Use case ID	ManageUseCase
Actors	Developer
Pre conditions	The developer has opened a project of his workspace.
Main flow of events	1. The use case begins when the user has opened one of his projects and clicks the “Add Use Case” button or the “View Use Cases” button. 2. If the developer clicks on the “Add Use Case” button, he can create a use case and specify the actors, preconditions, main flow and postconditions. 3. If the developer clicks the “View Use Cases” button: 3.1. He can view all of his use cases and update them to refine the system requirements. 3.2. He can delete a preferred use case at any time.
Post conditions	-

3.4 <Use Case 4>

Use case ID	ManageCRCCards
Actors	Developer
Pre conditions	The developer has opened a project of his workspace.
Main flow of events	1. The use case starts when user opens one of his projects and clicks the “Add CRC Card” button or the “View CRC Cards” button. 2. If the developer clicks the “Add CRC Card”, he can create a CRC Card by specifying its class name, responsibilities and collaborations and link it to use cases so that he can show how project classes support the use case behavior. 3. If the developer clicks the “View CRC Cards”, he can update/delete any CRC Card.
Post conditions	-

3.5 <Use Case 5>

Use case ID	GenerateDiagrams
Actors	Developer
Pre conditions	The project has use cases and/or CRC Cards.
Main flow of events	1. The use case begins when the developer clicks the “Scripts” button from a project of his. 2. The developer can generate scripts for the visualization of the use cases of a project as use case diagrams by clicking “Show Use Case Script”. 3. The developer can generate scripts for the visualization of the CRC cards of a project as class diagrams by clicking “Show CRC Script”.
Post conditions	-

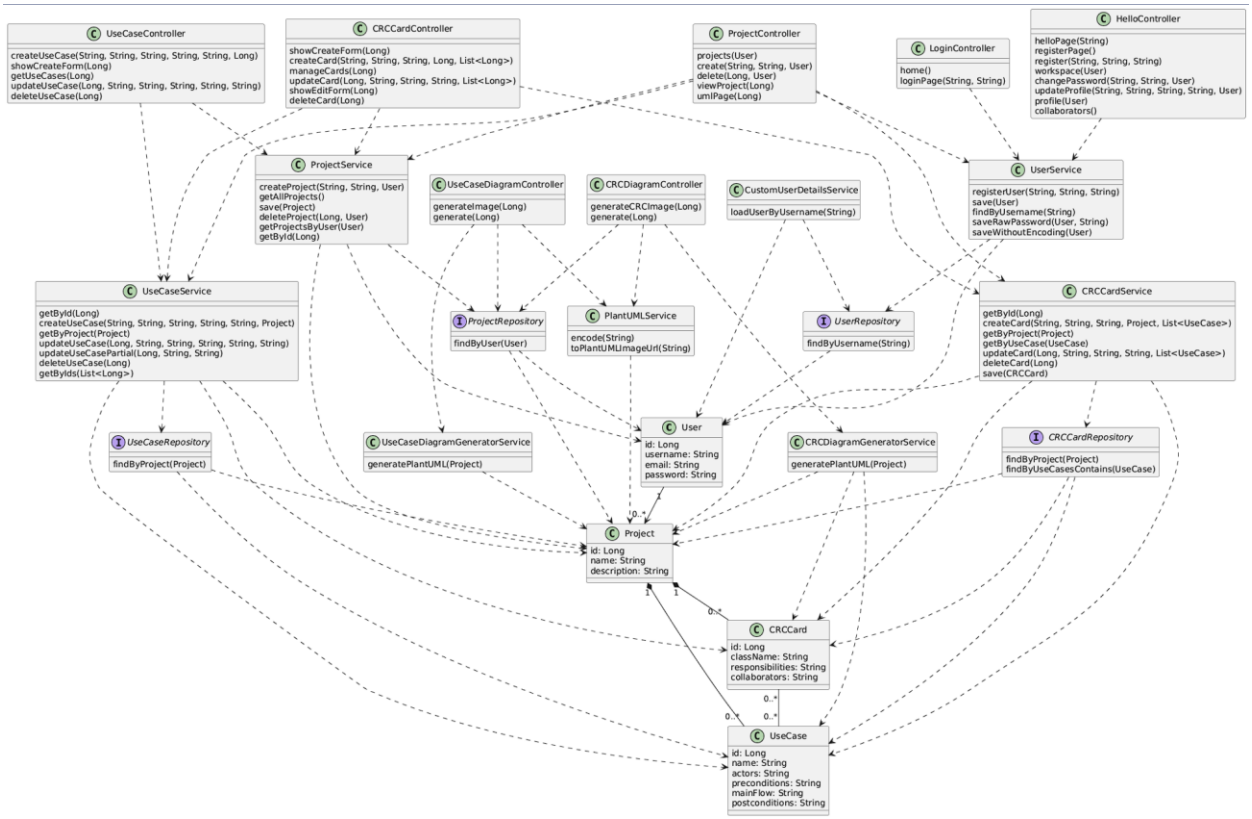
3.6 <Use Case 6>

Use case ID	Collaboration
Actors	Developer
Pre conditions	At least two users exist.
Main flow of events	1. The use case begins when the developer clicks the “Collaboration” button from the home page. 2. The developer can share a project with teammates. 3. The developer can write comments on use cases and CRC cards.
Post conditions	At least one of the users has shared a project for others to see and leave their comments.

4 Design

4.1 Architecture

UML Package Diagram



UML Class Diagram

4.2 Design

Class Name: User	
Responsibilities: ▪ Stores user credentials and profile information. Manages ownership of projects and authentication data.	**Collaborations:** ▪ HelloController

Class Name: Project	
Responsibilities: ▪ Stores project information ▪ Manages UseCases and CRCCards related to the project.	**Collaborations:** ▪ User ▪ UseCase ▪ CRCCard ▪ ProjectController

Class Name: UseCase	
Responsibilities: ▪ Stores use case information including actors, flows, preconditions, and postconditions.	**Collaborations:** ▪ Project ▪ CRCCard ▪ UseCaseController

Class Name: CRCCard	
Responsibilities: ▪ Represents CRC analysis information including responsibilities and collaborators of a class.	**Collaborations:** ▪ Project ▪ UseCase ▪ CRCCardController

Class Name: ProjectService	
Responsibilities: ▪ Handles business logic related to projects, project creation, retrieval, update, and deletion.	**Collaborations:** ▪ ProjectRepository ▪ Project ▪ User ▪ ProjectController

Class Name: PlantUMLService	
Responsibilities: ▪ Encodes PlantUML scripts and generates diagram image URLs.	**Collaborations:** ▪ UseCaseDiagramController ▪ CRCCardDiagramController

Class Name: UseCaseService	
Responsibilities: ▪ Manages creation, update, retrieval, and deletion of use cases.	**Collaborations:** ▪ UseCaseRepository ▪ UseCase ▪ Project ▪ CRCCard ▪ UseCaseController

Class Name: CRCCardService	
Responsibilities: ▪ Handles CRC card management and business operations.	**Collaborations:** ▪ CRCCardRepository ▪ CRCCard ▪ Project ▪ UseCase ▪ CRCCardController

Class Name: CRCDiagramGeneratorService	
Responsibilities: ▪ Generates PlantUML diagrams ▪ Formats CRC data including use cases ▪ Builds UML string output	**Collaborations:** ▪ CRCCard ▪ Project ▪ UseCase ▪ CRCCardDiagramController

Class Name: CustomUserDetailsService	
Responsibilities: ▪ Loads and authenticates user data by username ▪ Retrieves user credentials from database ▪ Converts User entity to Spring Security UserDetails	**Collaborations:** ▪ UserRepository ▪ User ▪ HelloController

Class Name: UserService	
Responsibilities: ▪ Saves user data ▪ Retrieves users by username ▪ Encodes and stores passwords ▪ Registers new users	**Collaborations:** ▪ User ▪ UserRepository ▪ HelloController

Class Name: UseCaseDiagramGeneratorService (*go to last page for a note)	
Responsibilities: ▪ Generates PlantUML use case diagrams ▪ Processes the project's use cases ▪ Creates actors and use case relationships, adds notes with pre-conditions, main flow, post-conditions ▪ Produces the UML text output	**Collaborations:** ▪ Project ▪ UseCase ▪ PlantUMLService ▪ UseCaseDiagramController

Class Name: Demo1Application	
Responsibilities: ▪ **Main Class, starts the SpringBoot Application**	**Collaborations:** ▪ -

Class Name: CRCCardController	
Responsibilities: ▪ Displays CRC card forms ▪ Creates, updates, deletes CRC cards and retrieves project CRC cards ▪ Associates CRC cards with use cases	**Collaborations:** ▪ CRCCardService ▪ UseCaseService ▪ ProjectService ▪ CRCCard ▪ UseCase ▪ Project

Class Name: HelloController	
Responsibilities: ▪ Registers users ▪ Changes passwords ▪ Updates profiles ▪ Displays workspace/profile pages ▪ Retrieves authenticated users	**Collaborations:** ▪ User ▪ UserService

Class Name: LoginController	
Responsibilities: ▪ Redirects users to login page ▪ Displays login page ▪ Handles login error and success messages	**Collaborations:** ▪ -

Class Name: ProjectController	
Responsibilities: ▪ Displays user projects ▪ Creates and deletes projects ▪ Views project details ▪ Retrieves authenticated user data ▪ Loads UML page	**Collaborations:** ▪ Project ▪ User ▪ ProjectService ▪ UserService ▪ UseCaseService ▪ CRCCardService

Class Name: SequrityConfig	
Responsibilities: ▪ Configures authentication and authorization ▪ Configures login/logout ▪ Defines password encoding	**Collaborations:** ▪ CustomUserDetailsService

Class Name: UseCaseController	
Responsibilities: ▪ Creates, updates and deletes the use cases ▪ Displays use case pages	**Collaborations:** ▪ Project ▪ UseCase ▪ UseCaseService ▪ ProjectService

Class Name: UseCaseDiagramController (*go to last page for a note)	
Responsibilities: ▪ Generates PlantUML scripts ▪ Generates use case diagram image URLs ▪ Retrieves projects for diagram generation ▪ Provides API endpoints for use case diagrams	**Collaborations:** ▪ ProjectRepository ▪ UseCaseDiagramGeneratorService ▪ PlantUMLService ▪ Project

*NOTE: Our project creates the uml script, which we run at https://www.planttext.com/ for visualization.

**NOTES:

- src/main/com/example/Demo1Application.java (line 27): we put the path of the browser we want to run.

String chromePath =

"C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe";

- src/main/resources/Application.properties (lines 4 and 5): we put the username and password of our local database.

spring.datasource.username

spring.datasource.password